PUC Minas

ARQUITETURAS E FUNDAMENTOS DE PIPELINES DE DADOS

Prof. Victor Sales Silva
victorssbh@hotmail.com

Apresentação



FORMAÇÃO ACADÊMICA	ATUAÇÃO NO MERCADO
Técnico em informática e redes – SENAI(2002)	**Estagiário Help Desk / Estagiário DBA / DBA / Analista de Business Intelligence – Take.Blip (2011 - 2017)**
	Especialista de Business Intelligence Financeiro – AlmavivA do Brasil LTDA (2017 - 2018)
Engenheiro de Computação – PUC Minas (2009 – 2014)	**Especialista em Business Intelligence – Montreal Informática LTDA (2018 - 2019)** • **Ferrous Resource do Brasil LTDA (adquirida pela Vale S.A.)**
MBA em Banco de Dados – CEFET-MG (2015 – 2017)	**Especialista em Dados / Tech Lead – A3Data Consultoria LTDA (2019 – 2025)**
	Especialista em Dados – GFT Technologies (2025 – atualmente)
	Professor na pós-graduação – PUC Minas (2021 – atualmente)

Sobre a disciplina

Ementa

- Arquiteturas de dados: Data Lake, Data Warehouse, Lakehouse e Data Mesh.
- Padrões de integração de dados: ETL, ELT, Change Data Capture (CDC) e Stream Processing.
- Ingestão de dados: APIs, bancos relacionais, NoSQL, arquivos e dados streaming.
- Cloud storage: S3, Azure Blob, GCS.
- Feature engineering, data quality engineering, feature stores.
- Orquestração de workflows de dados: Apache Airflow, Prefect, dbt e Azure Data Factory.
- Data lineage, catalogação e schema evolution.
- Aplicação de IA para engenharia de dados: auto-discovery de schemas, geração de documentação, detecção inteligente de anomalias e sugestão de transformações.
- Projeto prático: pipeline completo end-to-end com automação inteligente.

O que utilizaremos de ferramentas?

- Além de alguns conceitos que já foram ou podem ser abordados durante o curso, iremos utilizar:

  - Programação em Python e SQL
  - Processamento e integração de dados
  - Bancos de dados e fontes de dados
  - Armazenamento e Data Lake
  - Ferramentas de qualidade e transformação de dados
  - Ambientes de desenvolvimento e containers
  - Computação em nuvem (Microsoft Azure)
  - Ferramentas de Inteligência Artificial

Divisão de pontos

Atividade	Pontuação	Tipo de Atividade
Exercício 1 – Arquitetura de um Pipeline de Dados	10	Individual
Exercício 2 – Ingestão de Dados de Diferentes Fontes	10	Individual
Exercício 3 – Integração e Transformação de Dados	10	Individual
Exercício 4 – Implementação da Qualidade dos Dados	10	Individual
Exercício 5 – Feature Engineering para Inteligência Artificial	10	Individual
Exercício 6 – Automação e Orquestração de um Pipeline	10	Individual
Trabalho prático	40	Grupo

Orientações e dicas

- Para mantermos a organização das aulas, atividades e comunicação, vamos seguir algumas orientações:
  - **Todas as entregas de atividades devem ser realizadas exclusivamente pelo Canvas. Atividades entregues fora da plataforma não serão consideradas (e-mail, Microsoft Teams et. al)**.
  - Todas as tarefas possuem 1 (uma) semana de prazo de entrega, contado a partir da data de cadastro no Canvas referente à aula em execução. Atentem-se ao prazo.
  - Qualquer solicitação de ajuda deve ser feita diretamente via mensagem no Canvas.
  - A gravação de todas as aulas ficará disponível no Canvas assim que o arquivo for disponibilizado para o professor.
  - Se acompanhar a aula até o fim do horário, é garantida a entrega do exercício previsto para a próxima semana.

Na aula de hoje...

- Fundamentos e ciclo de vida dos pipelines de dados
- Data Warehouse, Data Lake, Lakehouse e Data Mesh
- Arquiteturas de dados para aplicações de IA
- ETL, ELT e processamento batch e streaming
- Organização das camadas de dados e armazenamento
- Preparação do ambiente
- Prática

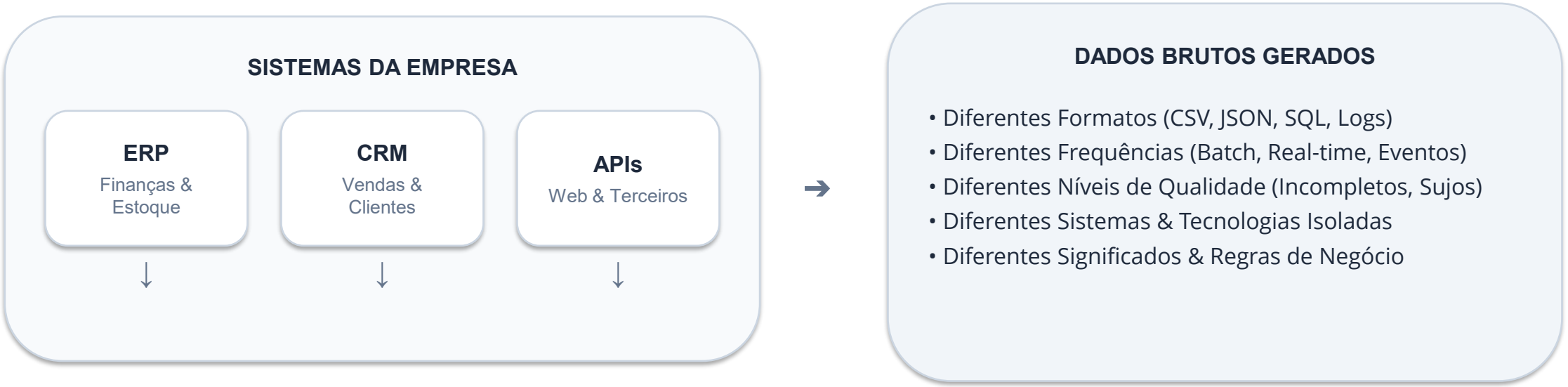
VAMOS COMEÇAR?

RESUMO

- Objetivos da aula
  - Fundamentos e ciclo de vida dos pipelines de dados
  - Data Lake, Data Warehouse, Lakehouse e Data Mesh
  - Arquiteturas de dados para aplicações de IA
  - ETL, ELT e processamento Batch e Streaming
  - Organização das camadas de dados e armazenamento
  - Preparação do ambiente de desenvolvimento
  - Construção de um pipeline de dados na prática

Fundamentos e ciclo de vida dos pipelines de dados

- Imagine uma empresa que possui:



Fundamentos e ciclo de vida dos pipelines de dados

- Os dados chegam:
  - em diferentes formatos;
  - em diferentes frequências;
  - com diferentes níveis de qualidade;
  - em diferentes sistemas;
  - com diferentes significados.

- **O desafio não é apenas armazenar dados.**
  - É transformar dados dispersos em dados utilizáveis.

Fundamentos e ciclo de vida dos pipelines de dados

- Um pipeline de dados conecta a origem ao consumo:



Fundamentos e ciclo de vida dos pipelines de dados

Fontes e ingestão de dados

1. Fontes de Dados

A origem de toda a cadeia de dados da organização:

- ✓ **Bancos Relacionais (OLTP):** PostgreSQL, MySQL, SQL Server.
- ✓ **APIs e Webhooks:** Integrações SaaS, serviços de terceiros.
- ✓ **Streaming & Logs:** Dispositivos IoT, logs de aplicação, navegadores.

2. Camada de Ingestão

Responsável pelo transporte confiável dos dados primários:

- ✓ **Modo Batch (Lote):** Extrações agendadas em intervalos definidos.
- ✓ **Modo Streaming:** Captura contínua de eventos em tempo real.
- ✓ **CDC (Change Data Capture):** Mapeamento imediato de alterações.

Fundamentos e ciclo de vida dos pipelines de dados

Armazenamento & Lakehouse

MinIO / Object Storage

Solução escalável para retenção de dados brutos e estruturados:

- ✓ Compatível com API S3 para ecossistema open-source.
- ✓ Suporte a volumes massivos não estruturados e semiestruturados.
- ✓ Baixo custo de armazenamento com alta disponibilidade.

Arquitetura em Camadas

Organização lógica do Data Lake para governança rigorosa:

- ✓ **Bronze (Raw):** Dados brutos exatamente como ingeridos da fonte.
- ✓ **Silver (Trusted):** Dados limpos, padronizados e enriquecidos.
- ✓ **Gold (Curated):** Dados agregados, validados e prontos para uso.

Fundamentos e ciclo de vida dos pipelines de dados

Transformação e Qualidade

Processamento com Spark

Motor distribuído para manipulação de volumes massivos:

- ✓ Tratamento de valores ausentes (Nulls) e remoção de duplicatas.
- ✓ Conversão de tipos de dados, datas e padronização de formatos.
- ✓ Enriquecimento e cruzamento entre diferentes tabelas.

Validação de Qualidade

Garantia de integridade antes da disponibilização final:

- ✓ Validação de esquemas e consistência de colunas.
- ✓ Regras de negócio e testes de completude de dados.
- ✓ Monitoramento contínuo contra divergências (Data Drift).

Fundamentos e ciclo de vida dos pipelines de dados

Consumo & Aplicações de IA

- A fase final entrega valor tangível para o negócio através de inteligência operacional:

  - **Modelos de Machine Learning:** Alimentação de algoritmos de treino e inferência com dados tratados.
  - **BI & Analytics:** Painéis estratégicos e relatórios executivos de alta confiabilidade.
  - **Sistemas Operacionais:** Realimentação de aplicações legadas via APIs analíticas.

Fundamentos e ciclo de vida dos pipelines de dados

- Um pipeline não termina quando o arquivo é gravado.



- Pense no pipeline como um processo contínuo.
  - Novos dados chegam → o pipeline processa → os dados são disponibilizados → o processo se repete.

Fundamentos e ciclo de vida dos pipelines de dados

Origem & Armazenamento Inicial

1. Produção

Geração contínua de eventos e registros operacionais nos sistemas de origem: bancos OLTP, APIs de parceiros, sensores IoT e logs de navegação.

2. Ingestão

Captação e movimentação confiável dos dados em lote (Batch) ou fluxo contínuo (Streaming) para o ecossistema analítico da empresa.

3. Armazenamento

Persistência dos dados brutos na camada Bronze (Raw Zone) do Data Lake ou Object Storage (ex: MinIO/S3) sem alteração da fonte.

Fundamentos e ciclo de vida dos pipelines de dados

Processamento & Validação

4. Preparação

Primeira higienização dos dados: desduplicação, conversão de formatos, tratamento de valores ausentes (Nulls) e estruturação do schema.

5. Transformação

Aplicação de regras de negócio complexas, cruzamentos (joins), agregações e criação de atributos (Feature Engineering) para tabelas Silver/Gold.

6. Validação

Testes automatizados de qualidade de dados: verificação de consistência, testes de nulos, regras operacionais e conformidade de tipos.

Fundamentos e ciclo de vida dos pipelines de dados

Consumo, Observabilidade & Loop

7. Consumo

Disponibilização dos dados validados para modelos de Machine Learning, painéis de BI executivos e reuso por APIs analíticas.

8. Monitoramento

Observabilidade ativa da saúde do pipeline: checagem de latência, volumetria, desvio de distribuição de dados (Data Drift) e falhas.

9. Reprocessamento

Ajustes, recargas históricas ou disparo automático de pipelines diante de alterações de schema ou retraining de modelos de IA.

Fundamentos e ciclo de vida dos pipelines de dados

- Em ambientes modernos de engenharia de dados, o pipeline não é uma execução isolada com início e fim definitivos:
  - **Dados em Mutação:** Hábitos de consumo, regras operacionais e fontes mudam constantemente.
  - **Realimentação (Feedback Loop):** O consumo gera insights e novos eventos que retroalimentam o sistema.
  - **Resiliência & Auto-Recuperação:** O monitoramento identifica falhas e dispara correções automaticamente.

Data Lake, Data Warehouse, Lakehouse e Data Mesh

- Conforme o volume e a variedade aumentam, surge uma pergunta:
  - Devemos colocar tudo em um banco de dados?
    - Nem sempre.

- Uma organização pode utilizar diferentes arquiteturas:
  - Data Warehouse
  - Data Lake
  - Lakehouse
  - Data Mesh

- Cada uma resolve problemas diferentes.

Data Warehouse, Data Lake, Lakehouse e Data Mesh

Data Warehouse

Características principais

- Dados prioritariamente estruturados
- Foco em BI, Relatórios e Consultas SQL
- Modelagem relacional (Star Schema / Snowflake)
- Dados limpos, validados e consistentes

cliente_id	cidade	vendas	mês
101	Belo Horizonte	R$ 125.000	08/2026
102	São Paulo	R$ 89.000	08/2026

ERP

CRM

Financeiro

→

ETL / ELT
Processamento e Modelagem

→

DATA WAREHOUSE
Relacional / SQL Analytics

↓

Dashboards / BI
Relatórios Executivos & SQL

Quando faz sentido?

Quando o principal objetivo é a análise estruturada, padronizada e consistente do negócio.

Data Warehouse, Data Lake, Lakehouse e Data Mesh

Data Lake

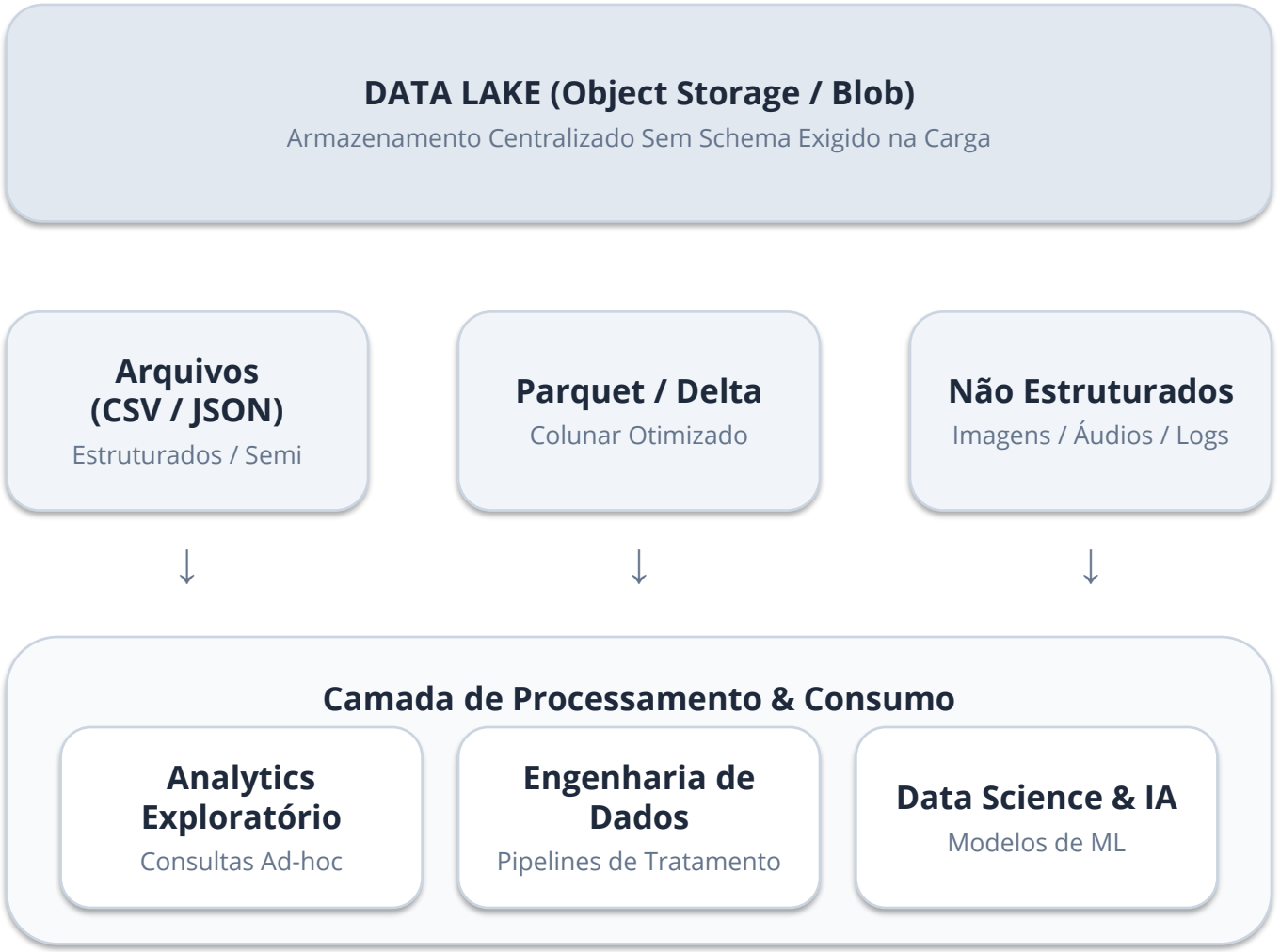
A Nova Lógica do Data Lake

Primeiramente armazenamos; depois decidimos como transformar e utilizar.

Formatos Suportados:

- Arquivos: CSV, JSON, Parquet
- Mídia: Imagens, Áudios, Vídeos
- Streaming: Logs, Eventos, IoT / Sensores

Destaque:
Ideal quando existe grande variedade de fontes, alto volume e necessidade de exploração sem restrições de schema prévio.



Data Warehouse, Data Lake, Lakehouse e Data Mesh

Data Lake

O Risco: Data Swamp (Pântano de Dados)

dados/ (Diretório Sem Governança)

```
├── arquivo1.csv
├── teste.csv
├── final.csv
├── final_final.csv
├── dados_novos.csv
└── dados_novos_corrigidos.csv
```

Um Data Lake NÃO significa 'jogar qualquer coisa em qualquer lugar'. Sem governança, ele rapidamente se transforma em um pântano inútil.

O Segredo do Sucesso: Governança & Organização

Armazenamento Flexível

Object Storage escalável e de baixo custo

Organização & Catálogo de Metadados

Mapeamento claro da origem e estrutura dos arquivos

Qualidade & Linhagem (Data Lineage)

Rastreabilidade e validações contínuas

Alerta AWS

A AWS destaca ativamente que a falta de governança é a causa #1 para a degradação de Data Lakes em Data Swamps.

Data Warehouse, Data Lake, Lakehouse e Data Mesh

Data Warehouse x Data Lake

CARACTERÍSTICA	DATA LAKE	DATA WAREHOUSE
Tipos de Dados	Estruturados e Não Estruturados	Principalmente Estruturados
Schema (Esquema)	Schema-on-Read (Mais flexível)	Schema-on-Write (Mais definido)
Caso de Uso Principal	Data Science, ML, Exploração	BI, Relatórios, Analytics Executivo
Armazenamento	Object Storage (S3, MinIO, Blob)	Engine Analítica Relacional
Flexibilidade	Alta flexibilidade	Menor flexibilidade (Mais rígido)
Usuários Típicos	Engenharia / Ciência de Dados	Analistas de BI / Negócio
Custo de Storage	Muito Baixo	Médio / Alto

Não existe um vencedor universal!

A escolha ideal depende do tipo de problema, do perfil da equipe e da maturidade dos dados da organização.

Data Warehouse, Data Lake, Lakehouse e Data Mesh

E se quisermos o melhor dos dois mundos?

Flexibilidade do Data Lake + Consultas SQL/BI + Governança do DW + Suporte a Machine Learning e IA

DATA LAKEHOUSE (Arquitetura Unificada)

Business Intelligence (BI)
Relatórios e Dashboards SQL

Machine Learning & IA
Treinamento de Modelos

Data Science & Analytics
Exploração Ad-hoc

↓ ↓ ↓

Camada Única de Dados Compartilhados, Governança & Transações ACID

Armazenamento em Object Storage (Delta Lake, Iceberg, Hudi)

Data Warehouse, Data Lake, Lakehouse e Data Mesh

Data Lakehouse



Por que esta organização incremental?

Refinamento Progressivo
O dado evolui de um formato bruto e heterogêneo até agregados otimizados para decisões.

Isolamento de Falhas
Se houver um erro na transformação da Silver ou Gold, não é preciso reingerir da fonte original.

Múltiplos Perfis de Consumo
Cientistas de dados usam Bronze/Silver; Analistas de BI utilizam Gold.

Governança Unificada
Rastreabilidade completa do ciclo de vida da informação (Data Lineage).

Data Warehouse, Data Lake, Lakehouse e Data Mesh

Data Lakehouse

BRONZE
(Raw Data)

Objetivo:
Preservar o dado exatamente como veio da origem.

Exemplos de Arquivos:
- pedido.csv
- cliente.json
- evento_log.json

Exemplo Prático:
pedidos_raw

SILVER
(Dados Tratados)

Objetivo:
Limpar, padronizar, integrar e validar os dados.

Transformações:
- Remoção de duplicatas
- Conversão de tipos de dados
- Validação de nulos e esquemas

Exemplo Prático:
pedidos_tratados

GOLD
(Dados Prontos)

Objetivo:
Agregar, enriquecer e orientar o dado ao consumo de negócio.

Características:
- Tabelas agregadas
- Regras de negócio aplicadas
- Otimizado para alta performance SQL

Exemplo Prático:
vendas_por_cliente

Data Warehouse, Data Lake, Lakehouse e Data Mesh

Data Lakehouse

- Por que separar em camadas?

Cenário de Exemplo: O Modelo de IA Apresentou um Resultado Inesperado!

Se os dados originais foram preservados na camada Bronze, a equipe não precisa 'adivinhar' o que deu errado. É possível rastrear o fluxo completo e reprocessar sem perda de informação:

[Bronze: Dado Bruto Origem] ➔ [Silver: Dado Limpo] ➔ [Gold: Agregado] ➔ [Dataset ML / IA]

1. Rastreabilidade & Auditoria

Permite voltar a qualquer etapa anterior e investigar problemas de dados.

2. Reprocessamento Seguro

Possibilita reprocessar regras de negócio sem precisar consultar os sistemas de origem novamente.

3. Separação de Responsabilidades

Engenheiros cuidam do Bronze/Silver; Analistas e Cientistas focam na Gold.

4. Controle de Qualidade

Garante que dados sem tratamento não cheguem aos dashboards da diretoria.

Data Warehouse, Data Lake, Lakehouse e Data Mesh

- E quando a organização cresce?

A Complexidade da Escala Organizacional

Financeiro	Marketing	Vendas	Logística	RH	Atendimento

Cada área produz, entende e domina os seus próprios dados de negócio. Centralizar TODO o atendimento em uma única equipe de dados cria um gargalo operacional gigantesco.

Surge uma nova abordagem organizacional e arquitetural:

DATA MESH

Descentralização da responsabilidade de dados por Domínios de Negócio

Data Warehouse, Data Lake, Lakehouse e Data Mesh

Data Mesh

Conceito Fundamental

No Data Mesh, os dados são tratados como PRODUTOS, mantidos e gerenciados diretamente pelas equipes de domínio que os produzem.

DATA PLATFORM (Infraestrutura Self-Service Centralizada)

Domínio Financeiro

Data Product:
'faturamento_mensal'

Domínio Vendas

Data Product:
'funil_conversao'

Domínio Marketing

Data Product:
'campanhas_cac'

↓ ↓ ↓

CONSUMIDORES (BI, ML, APIs, Outros Domínios)

Acesso Descentralizado com Governança Global

Data Warehouse, Data Lake, Lakehouse e Data Mesh

Data Lakehouse x Data Mesh

Data Lakehouse

Foco:
Arquitetura Tecnológica de Dados.

Resolve:
Como armazenar, processar e unir a flexibilidade do Data Lake com o suporte a ACID e SQL do Data Warehouse.

Data Mesh

Foco:
Modelo Organizacional, Domínio e Governança.

Resolve:
Como organizar equipes de dados para evitar gargalos em empresas em rápida expansão.

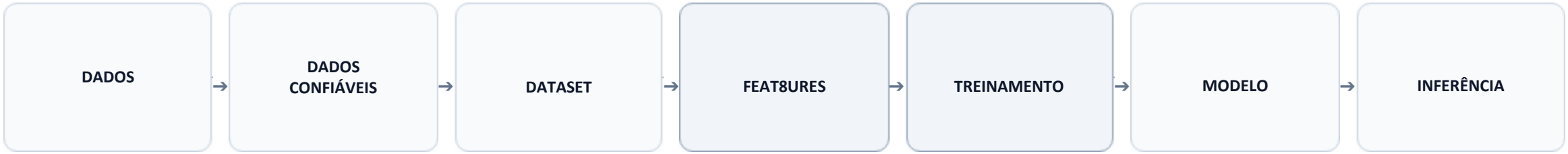
A Combinação Perfeita: Data Mesh + Lakehouse

Domínio Financeiro (Data Product A)
Implementado sobre um Data Lakehouse (Bronze/Silver/Gold)

Domínio Vendas (Data Product B)
Implementado sobre um Data Lakehouse (Bronze/Silver/Gold)

Arquiteturas de dados para aplicações de IA

- A IA começa nos dados!
- Um modelo de Machine Learning precisa de:



Uma arquitetura de IA precisa considerar o pipeline antes do modelo.

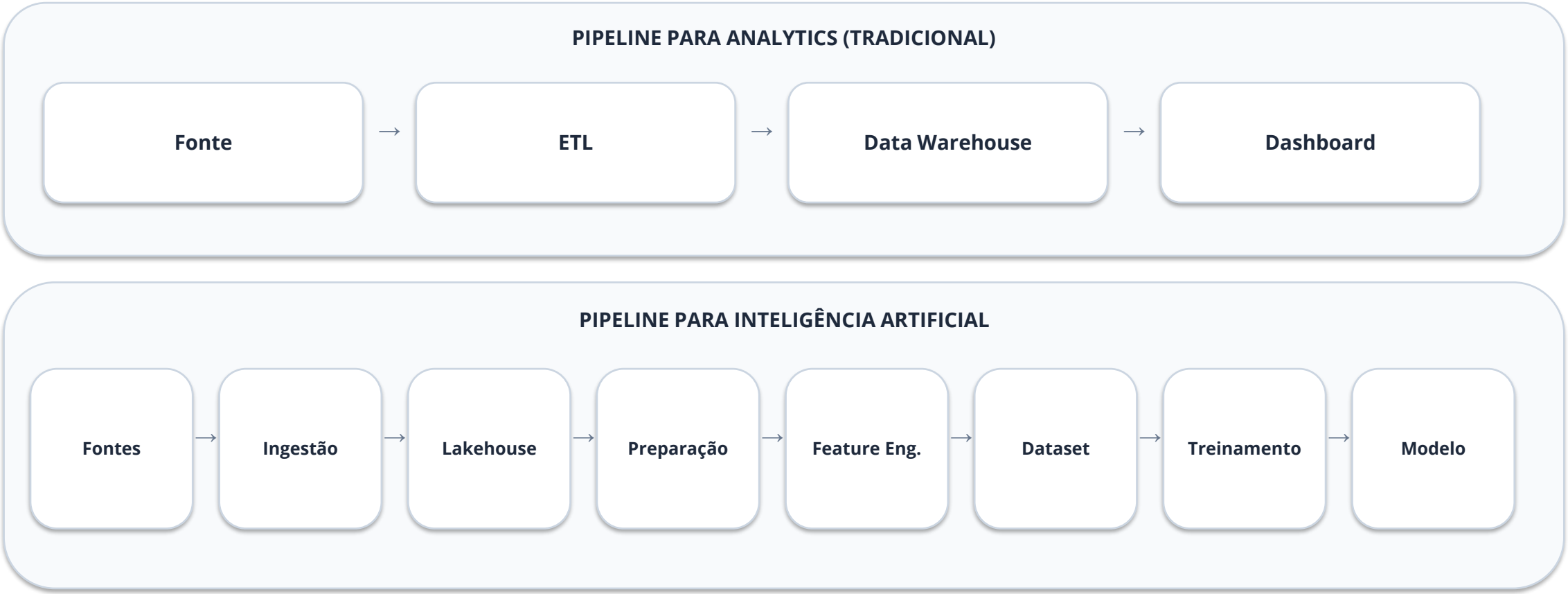
Arquiteturas de dados para aplicações de IA



Engenharia antes do Modelo:
Uma arquitetura de IA precisa garantir a integridade, reprodutibilidade e qualidade dos dados em cada etapa do pipeline antes de alimentar qualquer algoritmo de aprendizado.

Arquiteturas de dados para aplicações de IA

Pipeline tradicional x Pipeline para IA



O pipeline de IA precisa preparar dados para treinamento contínuo e inferência, não apenas para relatórios estáticos.

Arquiteturas de dados para aplicações de IA

Exemplo: Previsão de Churn

FONTES DE DADOS ORIGEM

CRM
- Cadastro de Clientes
- Histórico de Compras
- Registros de Atendimento

Sistema de Cobrança
- Faturas e Pagamentos
- Histórico de Inadimplência

Aplicação / Produto
- Logs de Acesso e Uso
- Frequência de Sessões

PIPELINE DE ENGENHARIA & ML

1. **Fontes (CRM, Cobrança, App)**
2. **Integração e Consolidação**
3. **Limpeza e Tratamento**
4. **Histórico por Cliente**
5. **Construção de Features**
6. **Dataset de Treinamento**
7. **Modelo de Churn**

Arquiteturas de dados para aplicações de IA

Exempl:o Previsão de Churn

- O dado precisa ter contexto:

PARA IA, NÃO BASTA O DADO BRUTO

Registros isolados e sem agregação temporal não fornecem padrões para o modelo:

- **cliente_id**
- **valor**
- **data**

PRECISAMOS CRIAR FEATURES (ATRIBUTOS)

Transformação de eventos em variáveis preditivas com significado de negócio:

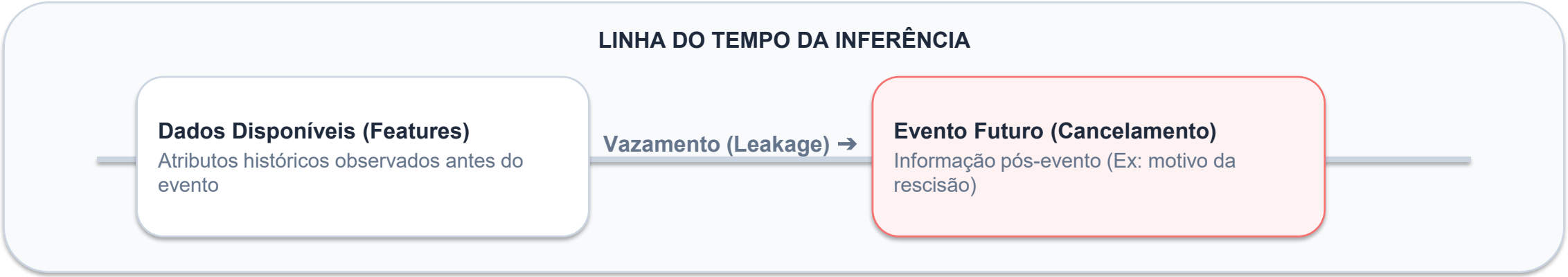
- ✓ **dias_desde_ultima_compra**
- ✓ **quantidade_compras_90d**
- ✓ **valor_medio_compras**
- ✓ **quantidade_chamados**
- ✓ **dias_desde_ultimo_acesso**

Papel da Engenharia de Dados:
O pipeline de dados transforma eventos e registros brutos em informações contextuais (features) prontas para o consumo do modelo.

Arquiteturas de dados para aplicações de IA

Exempl:o Previsão de Churn

- Perigo: Data Leakage
  - Ocorre ao utilizar, no treinamento, uma informação que só estará disponível DEPOIS do evento que queremos prever (ex: 'O cliente irá cancelar?').



IMPACTO DO DATA LEAKAGE NO NEGÓCIO

- O modelo 'vê o futuro' durante a etapa de treinamento e aprende correlações irrealistas.
- **Consequência: Excelente desempenho nas métricas de teste/treino, mas PÉSSIMO desempenho em produção real.**

ETL, ELT e processamento batch e streaming

ETL x ELT

ETL (Extract, Transform, Load)

Extract
Extração dos dados

↓

Transform
Transformação e limpeza

↓

Load
Carga no destino final

Os dados são transformados antes de chegar ao destino, garantindo apenas dados limpos na carga.

ELT (Extract, Load, Transform)

Extract
Extração dos dados

↓

Load
Carga bruta no destino

↓

Transform
Transformação no destino

Os dados são carregados primeiro e transformados posteriormente aproveitando a escala do destino.

ETL, ELT e processamento batch e streaming

- Um pipeline pode ser definido por **como os dados são transformados** e **como são processados ao longo do tempo**.

- **Como transformar os dados?**
  - **ETL — Extract, Transform, Load**
    - Os dados são extraídos, transformados e somente depois carregados no destino.
  - **ELT — Extract, Load, Transform**
    - Os dados são extraídos, carregados e transformados dentro do ambiente de destino.

- **Exemplo:**
  - ETL → tratar os dados antes de armazená-los no Data Warehouse.
  - ELT → carregar os dados no Lakehouse e transformá-los posteriormente.

ETL, ELT e processamento batch e streaming

- **Quando processar os dados?**

  - **Batch**
    - Processa um conjunto de dados acumulado em determinados intervalos.
  - **Streaming**
    - Processa os dados continuamente, à medida que são gerados.
  - **Exemplo:**
    - Batch → consolidar as vendas do dia.
    - Streaming → atualizar o indicador de vendas conforme cada venda acontece.

ETL, ELT e processamento batch e streaming

ESTRATÉGIA DE TRANSFORMAÇÃO	FORMA DE PROCESSAMENTO
ETL / ELT	Batch / Streaming
Define onde e quando ocorre a transformação	Define quando e continuamente como os dados são processados

Exemplo:
Um pipeline pode utilizar **ELT + Batch** para gerar um relatório diário ou **ELT + Streaming** para atualizar indicadores continuamente.

ETL, ELT e processamento batch e streaming

ETL x ELT

ATRIBUTO	ETL	ELT
Transformação	Antes do destino	Depois do carregamento
Dado Bruto	Nem sempre preservado	Mais fácil preservar
Flexibilidade	Menor	Maior
Uso Comum	Sistemas tradicionais	Plataformas modernas
Dependência	Engine de transformação	Engine de destino

Exemplo Prático ETL

CSV ➔ Python ➔ PostgreSQL

O arquivo é processado e transformado em memória antes do carregamento no banco relacional.

Exemplo Prático ELT

CSV ➔ Data Lake ➔ Spark/SQL ➔ Tratados

O dado bruto entra direto no Data Lake e a computação distribuída transforma sob demanda.

ETL, ELT e processamento batch e streaming

ETL x ELT

ETL pode fazer sentido quando:

- **Caso 1**
  O destino exige dados estritamente já transformados.
- **Caso 2**
  Existe uma etapa crítica de validação/segurança antes da carga.
- **Caso 3**
  O ambiente e a capacidade do destino são limitados.
- **Caso 4**
  Regras de negócio/privacidade precisam ser aplicadas antes do armazenamento final.

ELT pode fazer sentido quando:

- ✓ **Caso 1**
  Existe infraestrutura de armazenamento barato e altamente escalável.
- ✓ **Caso 2**
  Queremos preservar o dado bruto para auditorias e novas visões.
- ✓ **Caso 3**
  Temos engines analíticas poderosas no destino (Spark, DW Cloud).
- ✓ **Caso 4**
  Diferentes consumidores e áreas precisam dos mesmos dados brutos.

Decisão Arquitetural:
Não é uma questão de “qual é melhor”. Trata-se de alinhar a arquitetura aos requisitos de custo, compliance, poder computacional e uso do negócio.

ETL, ELT e processamento batch e streaming

ETL x ELT

ETL pode fazer sentido quando:

- **Caso 1**
  O destino exige dados estritamente já transformados.
- **Caso 2**
  Existe uma etapa crítica de validação/segurança antes da carga.
- **Caso 3**
  O ambiente e a capacidade do destino são limitados.
- **Caso 4**
  Regras de negócio/privacidade precisam ser aplicadas antes do armazenamento final.

ELT pode fazer sentido quando:

- ✓ **Caso 1**
  Existe infraestrutura de armazenamento barato e altamente escalável.
- ✓ **Caso 2**
  Queremos preservar o dado bruto para auditorias e novas visões.
- ✓ **Caso 3**
  Temos engines analíticas poderosas no destino (Spark, DW Cloud).
- ✓ **Caso 4**
  Diferentes consumidores e áreas precisam dos mesmos dados brutos.

Decisão Arquitetural:
Não é uma questão de “qual é melhor”. Trata-se de alinhar a arquitetura aos requisitos de custo, compliance, poder computacional e uso do negócio.

ETL, ELT e processamento batch e streaming

Processamento em Lote (Batch)

- No processamento Batch, os dados acumulados durante um período são processados em lotes definidos.

Fluxos Típicos de Execução

Janela Programada
00:00 ➔ 06:00 ▶ Processa Vendas ▶ Resultado

Janela fixa onde o processamento executa em segundo plano.

Recorrência Diária
Todo dia às 02:00 ↓ Processar vendas do dia anterior

Lote focado na consolidação periódica dos dados retroativos.

Casos de Uso Comuns do Batch

Fechamento Financeiro
Consolidação e balanços contábeis periódicos.

Atualização de Indicadores
Carga de Dashboards executivos e BI.

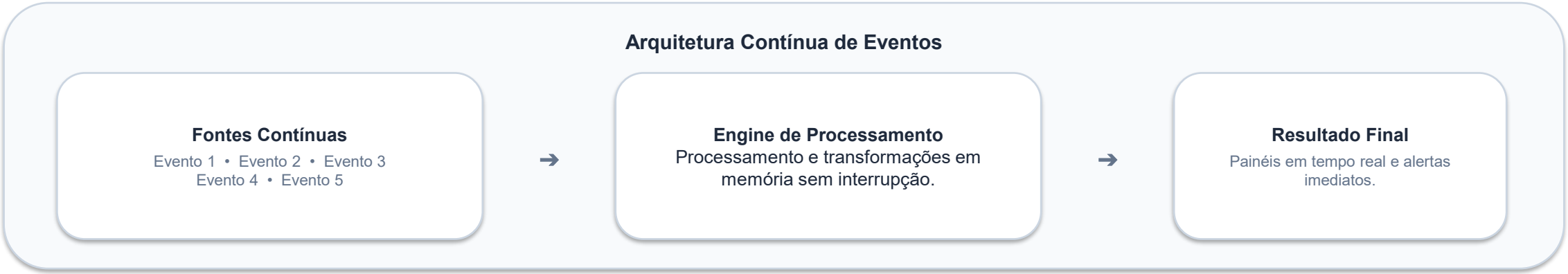
Geração de Relatórios
Emissão de relatórios fiscais ou operacionais.

Processamento de Arquivos
Cargas massivas noturnas de arquivos TXT/CSV.

ETL, ELT e processamento batch e streaming

Processamento em Tempo Real (Streaming)

- No Streaming, os dados são capturados, validados e processados continuamente à medida que chegam.





ETL, ELT e processamento batch e streaming

Batch x Streaming

ATRIBUTO	BATCH	STREAMING
Dados	Acumulados em blocos	Fluxo contínuo e ininterrupto
Execução	Periódica (agendada)	Contínua / quase contínua
Latência	Maior (minutos a dias)	Menor (milissegundos a segundos)
Complexidade	Menor suporte de infraestrutura	Maior gestão de estado e falhas
Exemplo Típico	Relatório diário e fechamentos	Monitoramento e alertas em tempo real

Pergunta Estratégica de Arquitetura:

O negócio realmente precisa de tempo real?

A implementação de arquiteturas em tempo real exige maior complexidade operacional, monitoramento e infraestrutura. Se o processo de tomada de decisão do negócio tolera atualizações periódicas, o modelo Batch continuará sendo a solução mais simples, robusta e custo-eficiente.

ETL, ELT e processamento batch e streaming

Quando usar Batch ou Streaming?

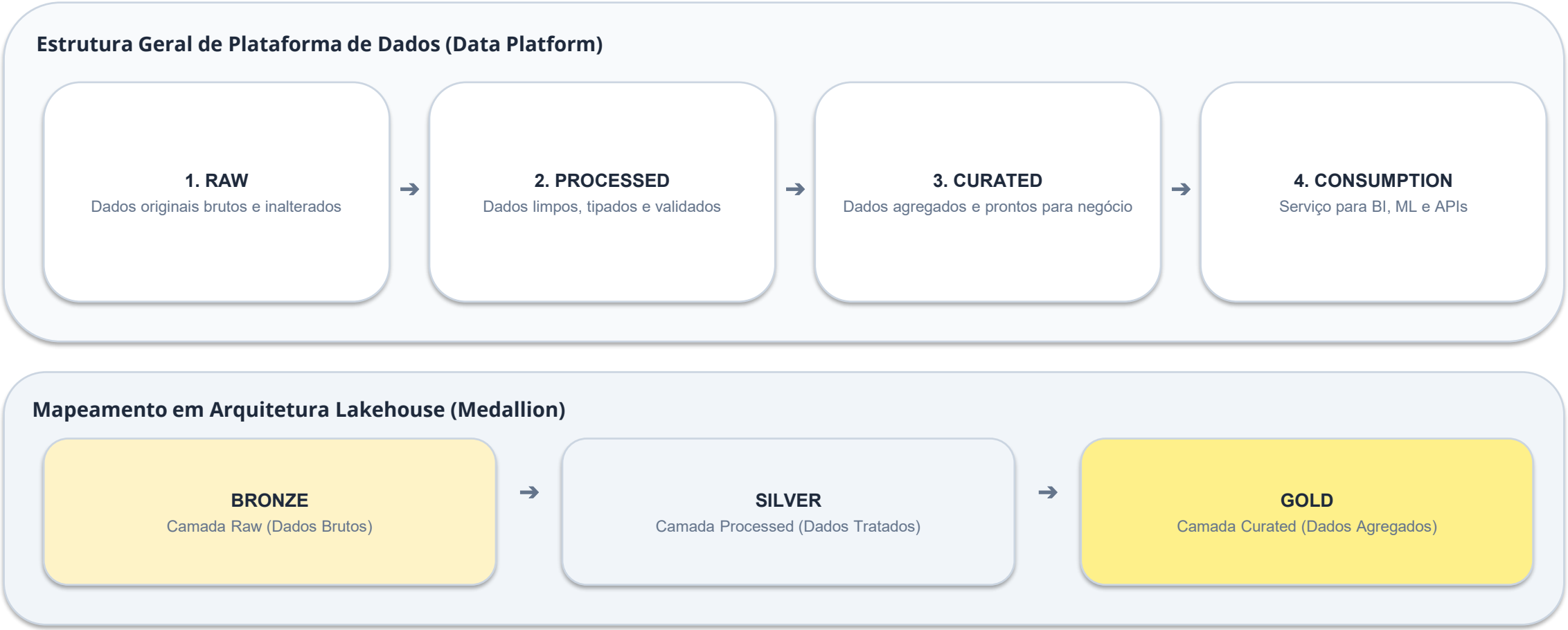
- Exemplo Prático: Como a necessidade de latência do negócio determina o modelo de processamento para o Monitoramento de Vendas.



Conclusão:
A escolha depende principalmente da latência exigida pelas decisões de negócio.

Organização das camadas de dados e armazenamento

- Uma arquitetura precisa de organização.



Organização das camadas de dados e armazenamento

Organização física no Data Lake

Exemplo de Estrutura de Diretórios (Data Lake)

data_lake_root/

```
data/
├── bronze/
│   ├── clientes/
│   ├── pedidos/
│   └── produtos/
├── silver/
│   ├── clientes/
│   └── pedidos/
└── gold/
    ├── vendas_cliente/
    └── indicadores/
```

Particionamento por Datas

Hierarquia de Partição

Os dados podem ser particionados em subdiretórios temporais hierárquicos para otimizar pesquisas:

ano/ ➔ mês/ ➔ dia/

Exemplo Prático:

```
pedidos/
└── ano=2026/
    └── mes=09/
        └── dia=08/
```

Essa estrutura permite que a engine analítica acesse diretamente a pasta do dia sem ler o histórico completo.

Organização das camadas de dados e armazenamento

Por que particionar dados?

Cenário Exemplo

Volume Total de Dados
Tabela de Vendas contendo 1 bilhão de registros armazenados.

Consulta Específica
Filtro aplicado: WHERE ano = 2026 AND mes = 08

Ganho de Performance
O motor de busca lê apenas a pasta 'ano=2026/mes=08', ignorando o restante do disco (Partition Pruning).

Decisão Arquitetural e Cuidados

Organização + Performance
Particionar evita a leitura desnecessária de grandes volumes de dados (I/O de disco) e reduz o custo computacional de consultas analíticas.

Atenção: 'Small Files Problem'
Mais partições NÃO significa automaticamente mais performance.

Particionar excessivamente cria arquivos muito pequenos, gerando sobrecarga de metadados na engine.

Regra de Ouro:
Particione por colunas de alta cardinalidade e uso frequente em filtros (ex: data), evitando criar partições minúsculas com poucos megabytes.

Organização das camadas de dados e armazenamento

Formato de dados

CSV (Linha por Linha / Texto)

Estrutura Bruta

```
id,nome,valor
1,Ana,100
2,João,200
```

✓ Vantagens

- Extremamente simples e legível por humanos.
- Suportado universalmente por qualquer sistema, planilha ou ferramenta de software.

✕ Desvantagens

- Alto volume de armazenamento sem compressão nativa.
- Lento e ineficiente para grandes consultas analíticas.

Parquet (Armazenamento Colunar)

Estrutura Colunar Binária

id ➔ [1, 2, 3, 4]
nome ➔ [Ana, João, ...]
valor ➔ [100, 200, ...]

✓ Vantagens

- Armazenamento por colunas extremamente compacto.
- Leitura seletiva de colunas e alta taxa de compressão.
- Perfeito para processamento analítico de grande escala.

✕ Desvantagens

- Formato binário (não legível diretamente sem ferramenta).
- Maior custo para escritas/modificações linha a linha.

Organização das camadas de dados e armazenamento

Qual formato usar?

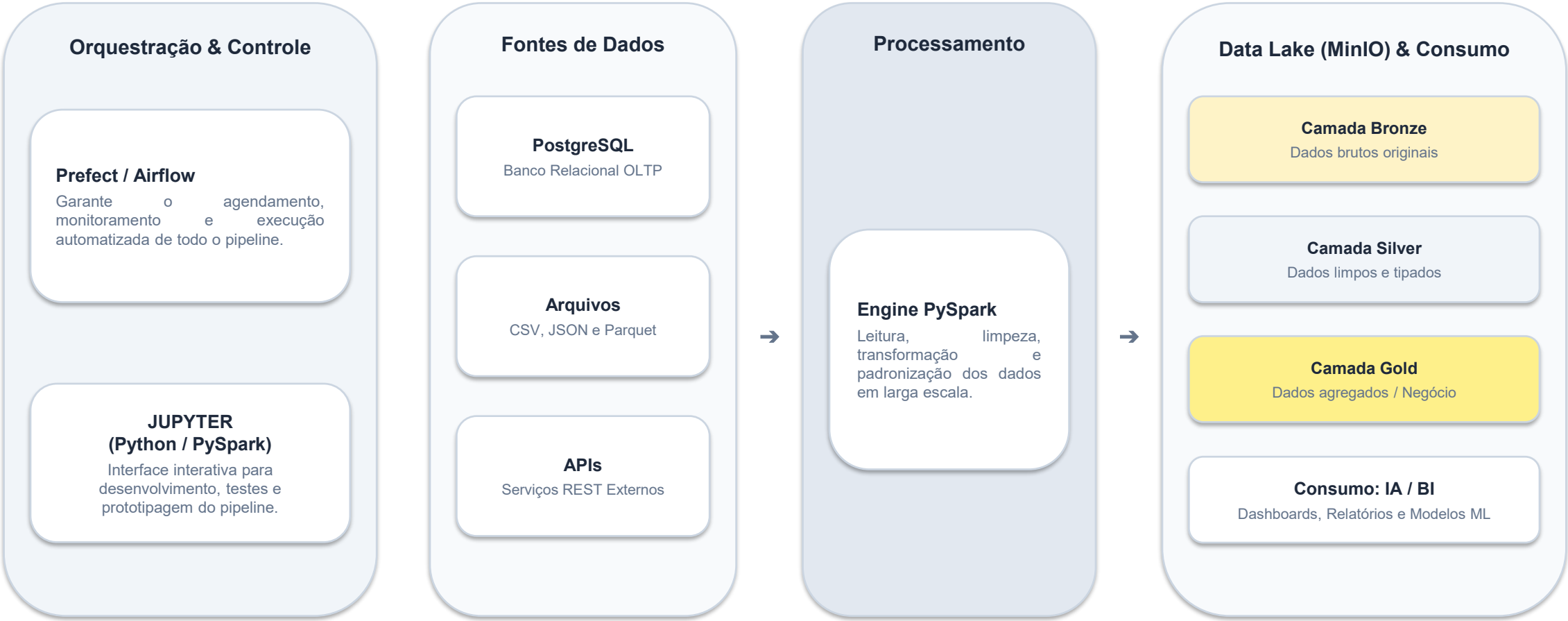
FORMATO	CARACTERÍSTICA PRINCIPAL	USO RECOMENDADO
CSV	Simples, texto plano e universal	Troca e exportação simples de dados
JSON	Semi-estruturado e flexível	Comunicação via APIs e eventos
Parquet	Colunar, binário e altamente eficiente	Analytics, Big Data e Data Lakes
Avro	Baseado em Schema com alta serialização	Integração contínua e Streaming (Kafka)

Aplicação Prática no Nosso Laboratório



Preparação do ambiente

Arquitetura local de execução



Preparação do ambiente

Ambiente e objetivos

Stack Tecnológica Utilizada

Python / PySpark
Linguagem base e motor de processamento distribuído

DBT / Great Expectations
Transformação relacional e validação de qualidade de dados

PostgreSQL / MinIO
Banco relacional OLTP e Data Lake Storage S3-compatible

Apache Airflow / Docker / WSL
Orquestração de pipelines, conteinerização e ambiente Linux

Objetivo da Plataforma Local

1. Receber Dados
Capturar dados heterogêneos vindos de bancos relacionais, arquivos e APIs.

2. Processar Dados
Executar limpezas, transformações de tipo e padronizações com PySpark e DBT.

3. Armazenar & Organizar
Persistir os dados estruturados em camadas (Bronze, Silver e Gold) no MinIO.

4. Executar Pipelines
Orquestrar e agendar o fluxo completo de ponta a ponta com resiliência.

Preparação do ambiente

Por que usar Docker?

Abordagem Tradicional (Sem Docker)

Instalação Direta no SO Host

Incompatibilidade de Ambiente:
Instalação direta no sistema operacional de múltiplos serviços:
• Python, Java, Spark, PostgreSQL, MinIO.

Gargalos Enfrentados:
• Conflitos de versões e dependências entre ferramentas.
• Configurações complexas e difíceis de reproduzir em outras máquinas.
• Poluição do sistema operacional hospedeiro.

Abordagem Moderna (Com Docker)

PostgreSQL
Container isolado

MinIO Lake
Container isolado

Jupyter Lab
Container isolado

Vantagens Arquiteturais

Resultado e Benefícios Práticos:

✓ Isolamento Total: Cada componente possui seu próprio ambiente de execução.
✓ Reprodutibilidade: O ambiente funciona exatamente da mesma forma em qualquer máquina.
✓ Zero Conflitos: Dependências do Spark/Java não afetam outras aplicações do sistema.

Preparação do ambiente

Nosso primeiro pipeline prático

1. Origem (CSV)
Entrada de dados brutos em formato texto sem tratamento.

➔

2. Engine (PySpark / DBT)
Leitura ➔ Tipagem ➔ Limpeza ➔ Transformação relacional.

➔

3. Destino (Parquet)
Conversão para formato colunar de alta performance.

➔

4. Storage (MinIO Lake)
Organização física nas camadas Bronze, Silver e Gold.

Próximo Passo: Automação & Agendamento

Orquestração com Apache Airflow / Prefect
Automatiza a execução sequencial do pipeline, gerenciando falhas, retentativas e disparos por tempo ou eventos.

DÚVIDAS?

OBRIGADO!

Referências bibliográficas

- KLEPPIN, Martin. **Designing data-intensive applications**: the big ideas behind reliable, scalable, and maintainable systems. Sebastopol: O'Reilly Media, 2017.

- KLEIN, Andreas *et al*. **Data engineering with Apache Spark**: material de referência para processamento distribuído e engenharia de dados. [S.l.: s.n.], 202-?

- REIS, Joe; HOUSLEY, Matt. **Fundamentals of data engineering**: plan and build robust data systems. Sebastopol: O'Reilly Media, 2022.

Referências bibliográficas

- APACHE SPARK. **Apache Spark documentation**: Spark SQL, DataFrames and Structured Streaming. [S.l.]: Apache Software Foundation, 2026. Disponível em: https://spark.apache.org/docs/latest/. Acesso em: 15 set. 2026.

- AWS. **Data Lake, Data Warehouse e Lakehouse**: materiais de referência sobre arquiteturas de armazenamento, analytics e integração de dados. [S.l.]: Amazon Web Services, 2026. Disponível em: https://aws.amazon.com. Acesso em: 15 set. 2026.

- DATABRICKS. **Lakehouse architecture**: arquitetura Medallion e integração entre engenharia de dados, analytics e Machine Learning. [S.l.]: Databricks, 2026. Disponível em: https://docs.databricks.com. Acesso em: 15 set. 2026.

- DATABRICKS. **Reference architectures**: pipelines de dados organizados por Source, Ingest, Transform, Query/Process, Serve, Analysis e Storage. [S.l.]: Databricks, 2026. Disponível em: https://www.databricks.com/architecture. Acesso em: 15 set. 2026.

- MICROSOFT AZURE ARCHITECTURE CENTER. **Big data architectures**: referência para arquiteturas modernas de Big Data, Data Lake e cenários de processamento de dados para IA. [S.l.]: Microsoft, 2026. Disponível em: https://learn.microsoft.com/azure/architecture/. Acesso em: 15 set. 2026.